Reset Early, Reset Often, Saturate the SAT Core

Martin S. Fixman[1], Fredrik Rømming[2], and Sean B. Holden[3]

University of Cambridge,
United Kingdom
[1]msf43@cl.cam.ac.uk [2]fr409@cl.cam.ac.uk [3]sbh11@cl.cam.ac.uk

Abstract

We improve PySATCoP performance by resetting the connection tableau often. This causes more and more varied SAT statements to be added to the SAT prover, which causes it to saturate faster. The resulting prover solves problems faster and in significantly fewer steps than SATCoP and other state-of-the-art tableau provers. [Add actual numbers here.]

Contents

1 Introduction **1**

2 Preliminaries **2**
2.1 Connection Calculus and Tableau Solving Methods . . . 2
2.2 SATCoP and Methods that use SAT Saturation in a Tableau . . . 3

3 Research Vehicle **4**
3.1 SATResetCoP Specification . . . 4
3.2 Implementation Details . . . 5
3.3 Python Implementation . . . 5
3.4 TPTP Benchmarking . . . 6

4 Experimental Evaluation **6**
4.1 Experiment Design . . . 6
4.2 Results . . . 7
4.2.1 Individual Results . . . 7
4.2.2 Comparative Results . . . 8
4.2.3 Soundness Verification: Problem Satisfiability . . . 9

5 Future Work **9**

6 Acknowledgements **9**

1 Introduction

Connection tableau provers are attractive because the core proof search is small and goal-directed. This is especially visible in leanCoP, where a competitive first-order prover can be implemented with very little machinery[10, 9]. The cost is that the prover has to make many local control decisions: which clause to start from, which connection to try next, when to backtrack, and when to restart.

These choices matter because connection proof search is not confluent. A locally valid connection can bind variables in a way that makes another branch impossible to close later. leanCoP's cut option shows that even a crude restriction on backtracking can make a large practical difference, although it loses completeness unless it is used inside a complete schedule[8].

SATCoP changes the objective slightly. Instead of using the tableau only as a proof object that must eventually close, it also uses the tableau as a source of ground clauses for an incremental SAT solver[15]. Once the accumulated ground clauses are propositionally unsatisfiable, the

prover can stop even if the current first-order tableau has not closed. This suggests a different view of tableau control: a good run is not only one that closes the tableau quickly, but one that generates useful ground instances quickly.

This paper explores that view by resetting the tableau aggressively. If the current bounded search stops adding new ground information to the SAT solver, we abandon it and start another search. This is incomplete as a standalone proof procedure, but useful when the goal is to saturate the SAT core fast. The rest of the paper describes the resulting prover, SATResetCoP, and compares it against leanCoP-style and SATCoP-style baselines on TPTP problems.

2 Preliminaries

2.1 Connection Calculus and Tableau Solving Methods

Connection tableaux and model elimination give a goal-directed alternative to the saturation style used by provers such as Vampire[16]. Instead of deriving clauses by proving repeatedly derive clauses from a global clause set until a contradiction is found or the search space saturates[1, 19], connection calculi instead try to close a tableau by repeatedly connecting the goal literals to complementary literals, either on the current path or in a fresh copy of an input clause[7, 11].

The standard clausal connection tableau has three proof-building inferences[7, 12].

Start Choose an input clause and make its literals open goals.

Extension Open a branch on a clause containing a complementary literal, and add the remaining literals as new subgoals. This might require a substitution of variables with predicates.

Reduction Closes the current goal against a complementary literal on the branch. Like the previous step, this might require adding a substitution.

The calculus is sound and complete with backtracking and iterative deepening, but it is not confluent: a locally valid connection can bind variables in a way that prevents later branches from closing[14].

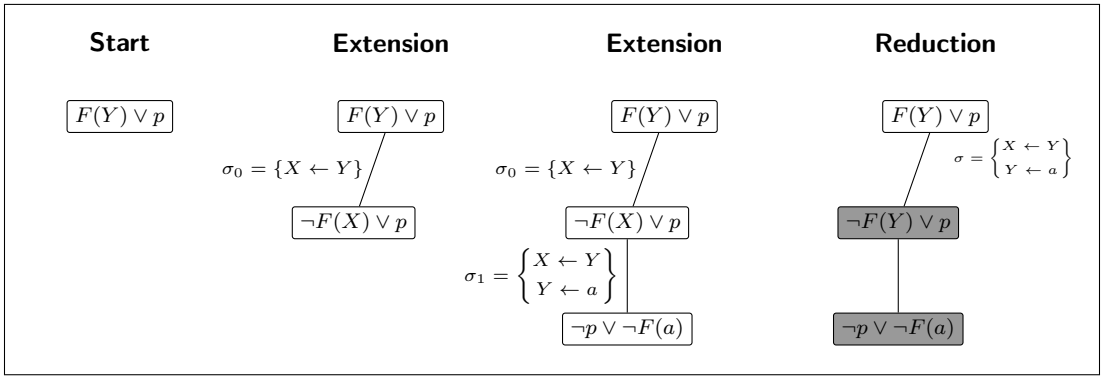


Figure 1: The three proof-building steps of a clausal connection tableau, taken from the tableau closure of SYN052-1.p[13]. The two Extension steps add an extra clause to the tableau; the Reduction step closes the nodes that were proven unsat. A full solution can be found in Figure 5.

Many different types of connection tableau provers were created around the years. leanCoP adapted this style of connection tableau provers to Prolog and became the base of a large family of provers for classical and non-classical logics[10, 9].

leanCoP comes with many optional predicate options. Most critical, cut commits a Prolog cut (!) after every successful Reduction and Extension. This removes many alternative local continuations and makes the base prover incomplete, but it also converges to a solution considerably faster[8]. Additionally, leanCoP iteratively deepens the maximum size of the tree when the options are saturated.

To keep the prover complete, the comp(I) option removes the maximum height restrictions and the cut when starting a tree that would have had maximum height I.

Later systems can be read as attempts to make this local control less crude. meanCoP separates cut on reductions from cut on extensions and introduces exclusive cuts, which preserve more alternatives than the inclusive cuts used by leanCoP[4]. hopCoP takes the opposite route: instead of deliberately throwing away alternatives, they learn constraints explaining why a search state is stuck, thereby reducing repeated backtracking while preserving completeness at a fixed iterative-deepening level[14].

Functional connection provers and the mlCoP and monteCoP line reorder proof steps using learned information or Monte Carlo Tree Search[5]. rlCoP and plCoP make this more explicit by treating proof search as a learning-guided control process over leanCoP-style actions[6, 22, 23]. The common theme is the one we use below: connection-tableau proving is naturally viewed as a sequence of small control decisions over proof actions, cuts, backtracking, and restarts.

2.2 SATCoP and Methods that use SAT Saturation in a Tableau

SAT and SMT solvers have long been used to support first-order reasoning, for example in finite model finding, instance generation, and splitting architectures[15, 18]. For a connection prover, the useful observation is simple: every clause that appears in a tableau is an instance of an input clause. If we ground these tableau clauses and collect them in a propositional SAT solver, then a SAT refutation of the collected ground clauses is a sound refutation of the original first-order problem[15].

Rawson and Reger implement this idea in SATCoP, a clausal connection-tableau prover based on leanCoP[15].

During backtracking search, clauses present in the tableau are heuristically grounded and added to an incremental SAT solver. If the SAT solver reports that the accumulated ground clauses are unsatisfiable, proof search can stop even if the first-order tableau itself has not been closed[15]. The remaining tableau variables must be mapped to ground terms before a clause is sent to SAT. Their implementation follows a Schulz-style heuristic: use the most frequent constant in the conjecture, and fall back to a fresh constant if no conjecture constant is available[15, 18, § 4.2].

Later modifications to SATCoP gave it control of the tableau using results from the SAT solver. In a later note included in the same paper, Rawson and Reger also describe a literal-selection refinement: prefer extension choices whose selected literal is true in the current SAT model[15, § 9.1].

The strategy used in this paper, SATResetCoP, pushes the ideas from SATcoP further by resetting trees as soon as the SAT model stops growing.

```
def solve_tableau(start_clauses):
  clauses := ground(start_clauses)

  max_depth := 1
  tableau := empty_tableau(max_depth)

  while solve_sat(clauses) != UNSAT:
    Actions := legal_actions(tableau)  // may include Backtrack

    model_rank(lit) :=
      0 if ground(lit) is true in SAT(clauses)
      1 if ground(lit) is unassigned in SAT(clauses)
      2 if ground(lit) is false in SAT(clauses)
      3 if lit = Backtrack

    a := argmin(a in Actions, model_rank(selected_literal_or_backtrack(a)))

    if a != Backtrack:
      step(a)
      clauses.add(ground(tableau))
      continue

    if ground(tableau) adds a new clause:
      tableau := empty_tableau(max_depth)
    else:
      tableau := empty_tableau(max_depth + 1)
      max_depth := max_depth + 1
```

Listing 1: Pseudocode of the SATResetCoP control policy.

3 Research Vehicle

3.1 SATResetCoP Specification

We propose implementing a new tableau prover, based on SATCoP, but with the strategy of attempting to saturate the SAT prover rather than to close the tableau.

Successive experimentation resulted in a simple strategy for achieving this: aggressively resetting the tree when the SAT solver stops adding new SAT entries results in a larger quantity and variety of SAT statements added to the solver in a smaller amount of time, which results in reaching a SAT unsatisfiability proof earlier.

SATResetCoP uses the same techniques for grounding as PySATCoP. It changes its tableau guidance to prioritise resetting the tableau often. The policy is presented in pseudocode in Section 3.1, where the ordering of forward actions follows the SAT-model literal selection idea of Rawson and Reger[15, § 9.1].

Just like PySATCoP, this approach is sound. While it is less complete than PySATCoP and other leanCoP variants, the speed at which it proves unsatisfiability of first-order problems can result in a larger amount of problems solved in a bounded amount of time or steps. It might be useful to incorporate SATResetCoP into the initial steps of a more complete schedule.

3.2 Implementation Details

The implementation keeps the active tableau search separate from the SAT state. A Reset clears the tableau, the current substitution, and the proof sequence, but it does not clear the ground clauses already given to the SAT solver. This means that every new bounded tableau attempt starts from an empty tree, while the SAT solver keeps the ground information produced by earlier attempts.

The grounded clauses used by PySATCoP and SATResetCoP come from clauses that have appeared in the tableau, after applying the tableau substitution accumulated so far. Start clauses are added to the SAT state when the run is initialised. After successful Extension and Reduction steps, the implementation grounds the clauses currently present in the tableau and records the genuinely new ones. These new clauses are then flushed to the SAT solver at reset points. The solver is used incrementally, so new clauses are added to the same CaDiCaL instance rather than rebuilding the SAT problem each time[2].

The grounding scheme follows the soundness argument of Section 2.2: the SAT solver receives only ground instances of clauses generated by the tableau. Ground terms are kept as they are, and variables assigned by the current tableau substitution are replaced by their assigned terms. Only tableau variables that remain unassigned need to be replaced by a ground term. Rawson and Reger follow the technique by Schulz[18] and use the most frequent constant in the conjecture, with a fresh constant as a fallback when the conjecture contains no constants[15, § 4.2]. SATResetCoP always uses this fallback-style choice, mapping every remaining tableau variable to the same fresh artificial constant. This gives a grounding scheme that is simpler than SATCoP, while keeping the same soundness condition.

The reset policy is the main implementation difference between SATResetCoP and PySATCoP. At a dead end, SATResetCoP checks whether the last tableau attempt produced any new ground clauses. If it did, the prover resets at the same depth, allowing another attempt to use the larger SAT state. If it did not, the prover resets at the next depth instead of spending more steps on local backtracking. The process stops as soon as the accumulated ground clauses become propositionally unsatisfiable, as in SATCoP[15].

SATResetCoP also follows SATCoP in randomising the Start and Extension steps. When starting a new tree, the implementation shuffles the possible starts and the literals inside each start clause. When several extensions are possible from the leftmost open node, it shuffles the extension choices and the literal order inside the copied extension clause. Reductions are not randomised in the current implementation. The random generator is seeded, so the experiments remain reproducible.

3.3 Python Implementation

As a testbed for out experiments we use the Connections Prober by Rømming et. al[17], which provides an implementation of leanCoP-style provers.

The connections prover makes it easy to implement different strategies to guide the tableau proof. This makes implementing the guidance of SATResetCoP simple and its code easy to read and evaluate.

The source code for this implementation is present in `https://github.com/mfixman/resetsatcop`, and is released under the GNU Lesser General Public Licence[3].

3.4 TPTP Benchmarking

Along with out implementation of SATResetCoP, PyCoP allows us to reimplement PyLeanCoP and PySATCoP in its own backend. This provides a fairer baseline for comparison.

We run SATResetCoP along with our versions of these two connection tableau strategies, along with a strategy that chooses the first possible action as control, on the 8344 CNF problems and 9091 FOF problems of all 50 categories containing at least one of these problems from the Thousands of Problems for Theorem Provers (TPTP) library[20].

The library provides a varied set of problems that's a standard way to prove first-order provers. It also provides declared statuses for these problems, which are a useful sanity check for false unsatisfiability results.

4 Experimental Evaluation

4.1 Experiment Design

The connections prover provides with PyCoP, a simple prover that chooses the first legal non-backtrack action at each point and resets the tree with an increased maximum height saturated.

As a second control policy we use our implementation of leanCoP[8], PyLeanCoP. This is a Python version of the leanCoP Prolog implementation, with the two recommended predicates used in the first step of the schedule by Otten[8, § 5.5].

`cut` Choose one choice from each extension and reduction and backtrack afterwards, in a similar way to to Prolog's `!` operator.

`comp(7)` The prover does iterative deepening of height 1 at a time when the tree is saturated until reaching maximum height 7. Afterwards limits on height are restricted.

These kinds of simple policies tend to be very effective in tableau provers[9].

Model	Explanation	Cut	UNSAT Proof	SAT Guidance	Reset on Backtrack
PyCoP PyLeanCoP	Simple control policy leanCoP with initial set of predicates.	✓			
PySATCoP_0	Based on the initial implementation of SATCoP[15].	✓	✓		
PySATCoP	Improvements on SATCoP[15, § 9.1].	✓	✓	✓	
SATResetCoP	Novel model, specified in Section 3.1.	✓	✓	✓	✓

Table 1: The four control models and the test model used in this experiment.

The TPTP library contains 8344 CNF problems and 9091 FOF problems separated in 50 categories[20]. Each problem/model pair is run for a maximum of 90 seconds and 1 million steps, on a single core of the University of Cambridge CSD3 Cluster with [add CPU information] and 3370 MiB RAM per core[21].

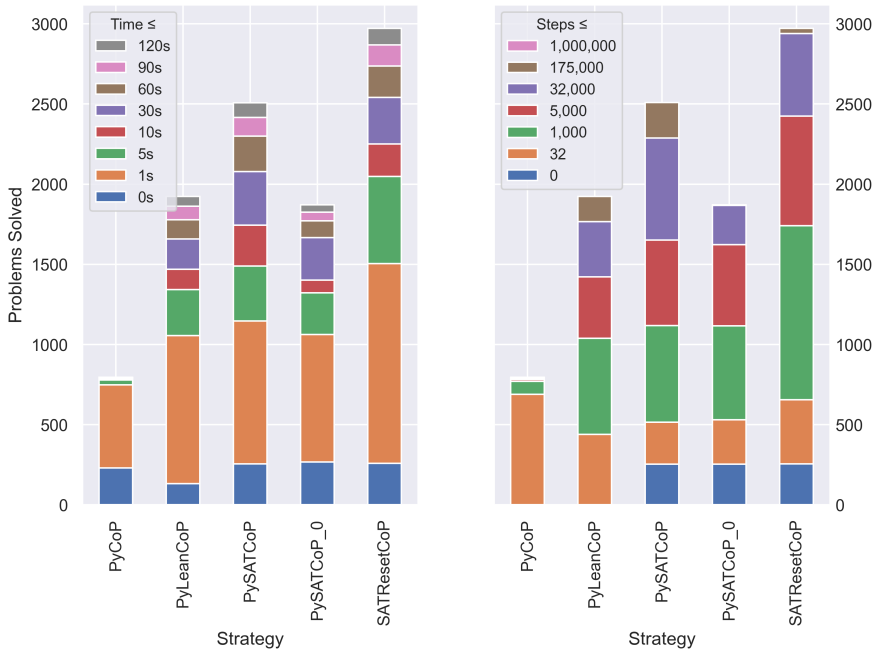
4.2 Results

4.2.1 Individual Results

SATResetCoP considerably improves the amount of problems solved by both $\mathsf{PySATCoP}_0$ and PySATCoP under the 90 second time limit, as shown in Table 2. This is mostly caused by the SAT solved reaching unsatisfiability in fewer time and steps than in previous models, as seen in Figures 2a and 2b.

Model	CNF	FOF	Total
PyCoP	205	587	792
PyLeanCoP	701	1155	1856
$\mathsf{PySATCoP}_0$	787	1042	1829
PySATCoP	1027	1396	2423
SATResetCoP	1227	1634	2861

Table 2: Amount of CNF and FOF problems solved within the time constraint by each model.



(a) Problems solved, stacked by time in seconds.

(b) Problem solved, stacked by amount of steps. SAT models can solve a problem in 0 steps if the initial grounding produces is SAT-unsatisfiable.

Figure 2: Results of running the models on all problems solved in a 90 second and 1 million step limit.

The 1 000 000 step limit is rarely reached, and most unsolved problems hit the 90 second limit instead.

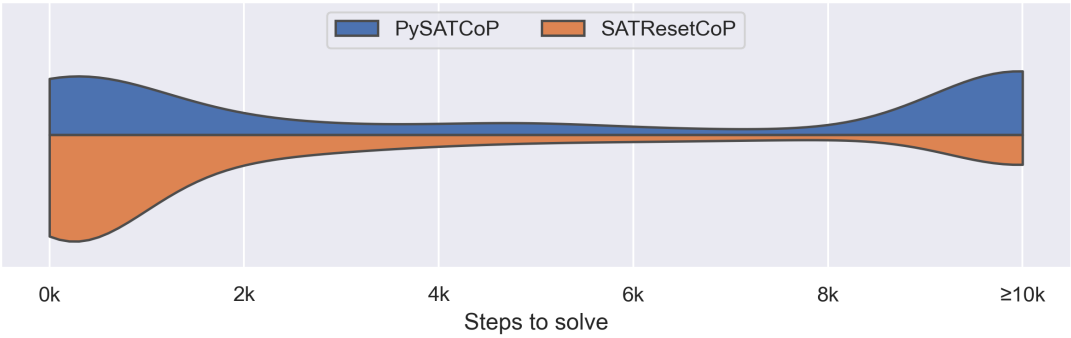


Figure 3: Steps taken by SATResetCoP to solve each problem within each TPTP problem type.

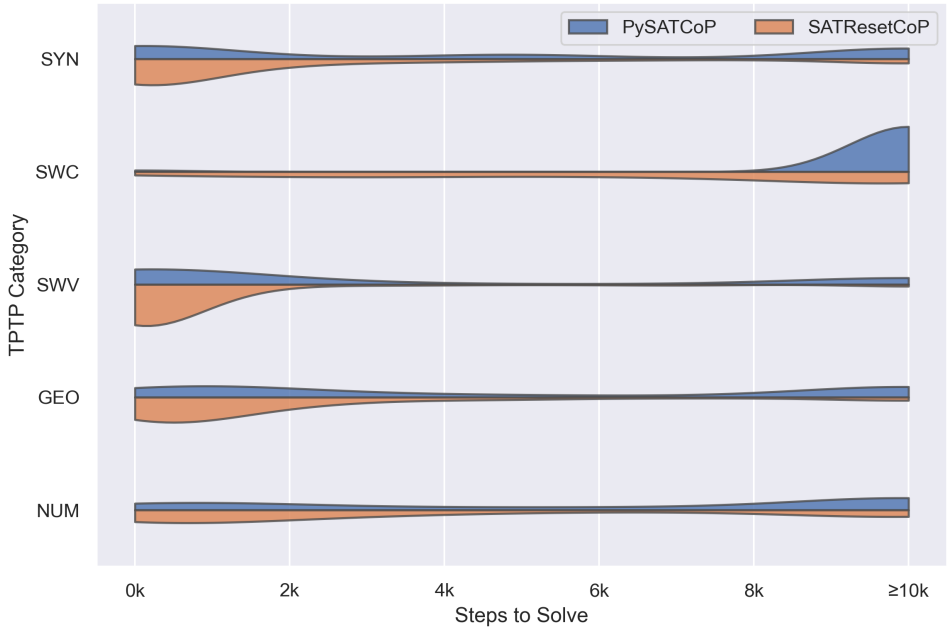


Figure 4: Density of problems solved at a certain amount of steps by PySATCoP and SATResetCoP to on categories SYN, SWC, SWV, GEO, and NUM of TPTP. The area of each violinplot is the same, and the plot shows how, besides solving more problems in each of these category, PySATCoP solves a significant amount in fewer steps. Unsolved problems are accumulated the $\geq 10k$ category on the right.

4.2.2 Comparative Results

The SAT guidance in SATResetCoP causes it to solve several problems that are unsolved by either models which do not have the same SAT saturation prover, and even by PySATCoP_0 and PySATCoP, which do not have similar efficient guidance. A comparative result can be seen in Table 3.

Model	CNF	FOF
PyCoP	205	587
PyLeanCoP	502	693
PySATCoP$_0$	184	129
PySATCoP	188	189
SATResetCoP	213	228

Table 3: Problems solved by each model and not solved by any preceding model in the table. [What's the point of separating CNF and FOF here? Maybe finding interesting TPTP types would be better.]

	FOF		CNF	
Declared status	Unsolved	Solved	Unsolved	Solved
Satisfiable	350	0	1094	0
CounterSatisfiable	732	0	0	0
Unknown	94	0	745	0
Open	4	0	28	0
Unsatisfiable	100	72	4496	1433
ContradictoryAxioms	204	39	0	0
Theorem	4426	1823	0	0

Table 4: Amount of problems solved and unsolved by SATResetCoP, grouped by TPTP declared status. No satisfiable or countersatisfiable problem was incorrectly proven to be unsatisfiable.

SATResetCoP is specially strong in problems which can be solved by small unsat cores. For example, Figure 5 shows the solution to the TPTP problem SYN052-1[13].

Figure 7 shows the amount of problems solved one of the models and not a previous one.

The size of the improvement between SATResetCoP and the next best model is notable.

[Add more analysis here, including size of unsat cores]

4.2.3 Soundness Verification: Problem Satisfiability

The problems in the TPTP library have a helpful specification of the satisfiability of different types of problems[20]. This is useful as a sanity-check: a sound solver should never "prove" unsatisfiability on a satisfiable problem, and their authors should be wary when proving unsatisfiability on an open or unknown problem. Note that this is not a way to prove soundness of the solver, but to check that it does not exhibit false unsatisfiability results on problems declared satisfiable or countersatisfiable.

We make a second run of SATResetCoP with a longer 10-minute limit, whose results can be seen in Table 4. These results do not show any negative results.

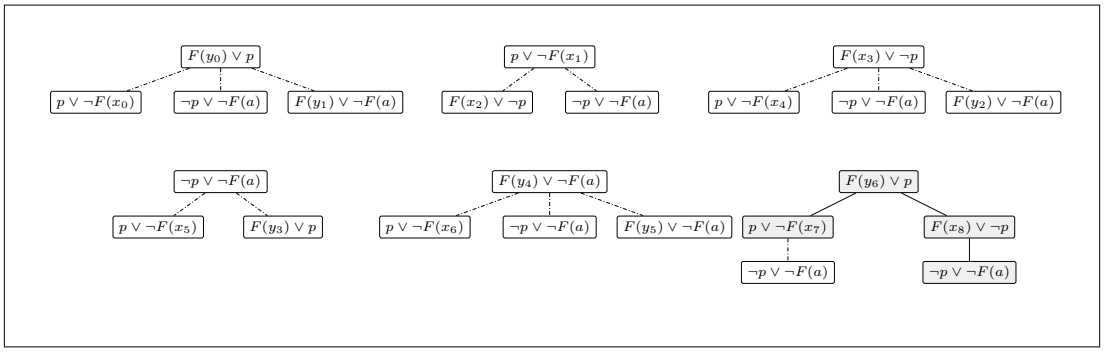
5 Future Work

6 Acknowledgements

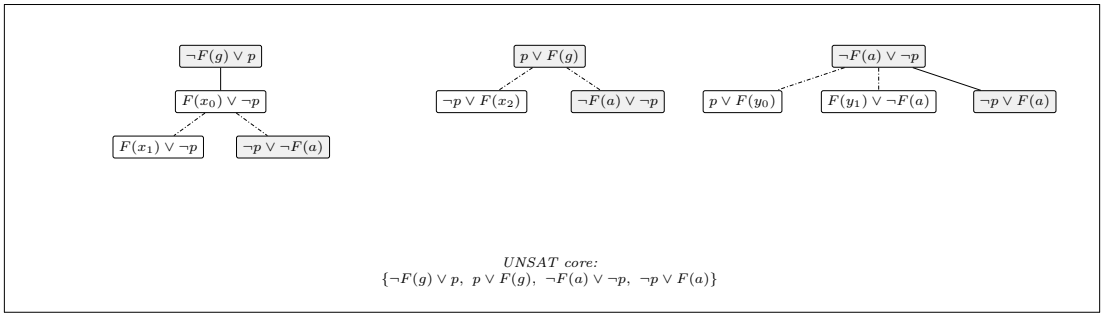
This work was performed using resources provided by the Cambridge Service for Data Driven Discovery (CSD3) operated by the University of Cambridge Research Computing Service (www.

$$p \vee \neg F(X) \qquad F(X) \vee \neg p$$
$$F(Y) \vee p \qquad \neg p \vee \neg F(a) \qquad F(Y) \vee \neg F(a)$$

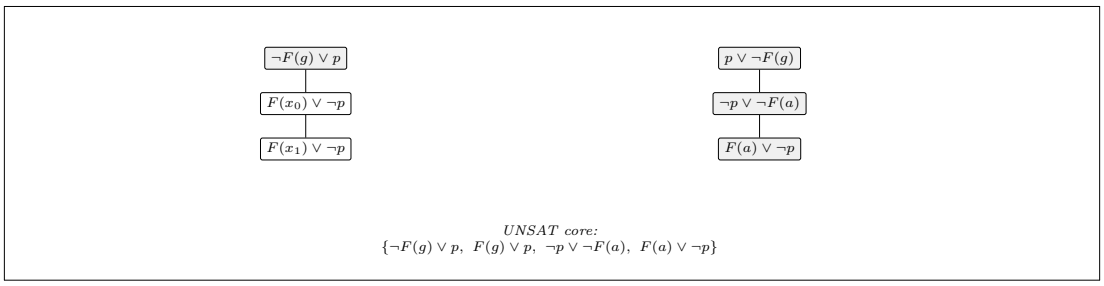
(a) Problem SYN052-1.p[13], adapted from https://tptp.org/TPTP/Problems/SYN/SYN052-1.p.



(b) Solution found by PyLeanCoP in 43 steps.



(c) Solution found by SATCoP in 34 steps.

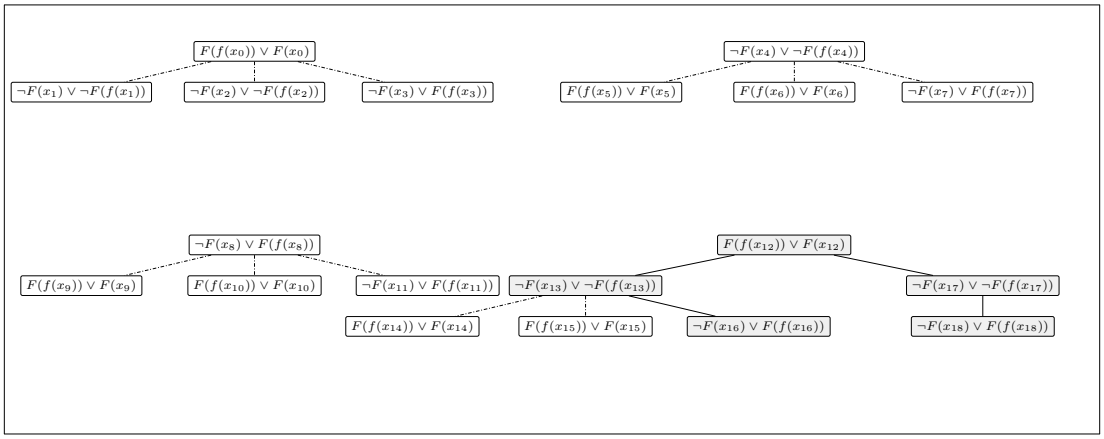


(d) Solution found by SATResetCoP in 11 steps.

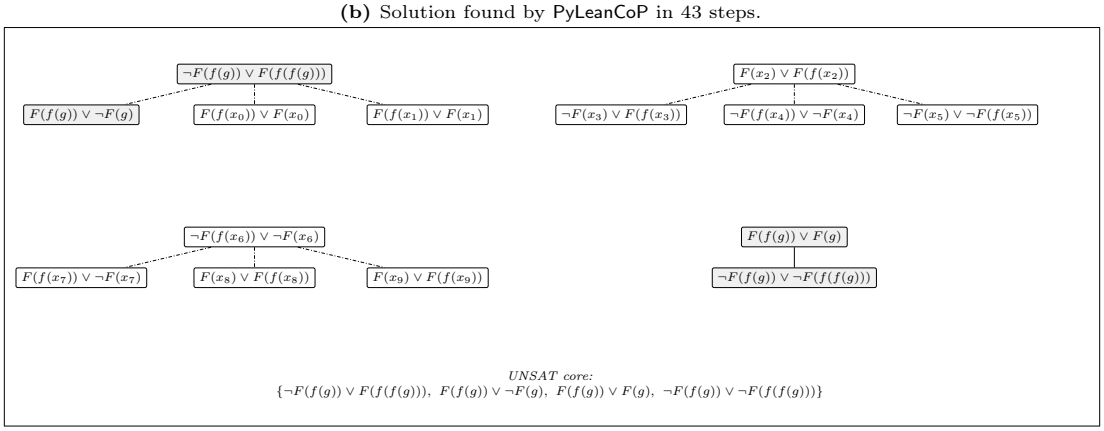
Figure 5: The unsatisfiability proof to the TPTP problem SYN052-1[13] coming from three provers: PyLeanCoP, PySATCoP, and SATResetCoP.

$$\neg F(X) \vee \neg F(f(x)) \qquad F(f(X)) \vee F(x)$$
$$\neg F(X) \vee F(f(X))$$

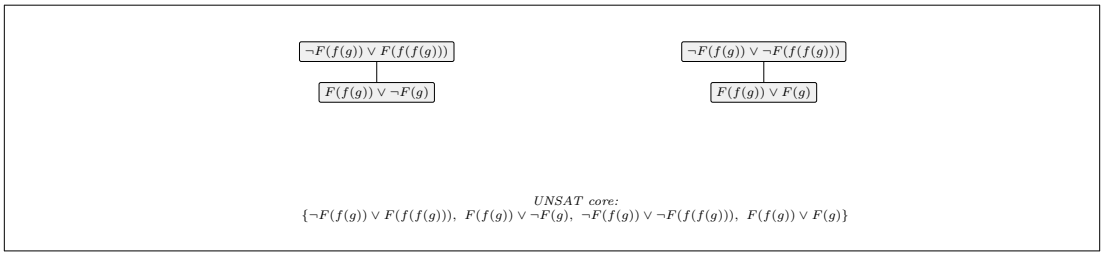
(a) Problem SYN081-1.p[13], adapted from https://tptp.org/TPTP/Problems/SYN/SYN081-1.p.



(b) Solution found by PyLeanCoP in 43 steps.



(c) Solution found by SATCoP in 34 steps.



(d) Solution found by SATResetCoP in 11 steps.

Figure 6: The unsatisfiability proof to the TPTP problem SYN081-1[13] coming from three provers: PyLeanCoP, PySATCoP, and SATResetCoP.

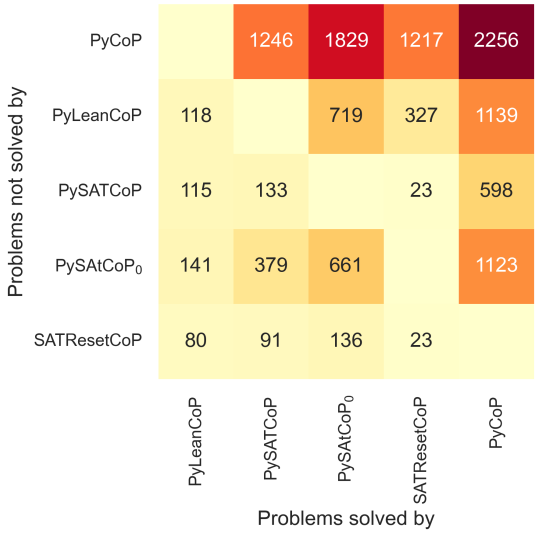


Figure 7: Pairwise comparison of model results: the cell in column y and row x shows the amount of problems solved by the model on y, but not the model in x. For example, SATResetCoP solved 1056 TPTP problems that PySATCoP$_0$ did not.

Model	SYN	SWC	SWV	GEO	NUM	Rest	Total
PyCoP	231	30	129	28	9	368	795
PyLeanCoP	486	14	163	216	86	958	1923
PySATCoP$_0$	459	2	161	177	90	982	1871
PySATCoP	549	11	173	260	151	1365	2509
SATResetCoP	672	64	215	293	179	1548	2971

Table 5: Amount of problems solved, per category, for each model.

csd3.cam.ac.uk), provided by Dell EMC and Intel using Tier-2 funding from the Engineering and Physical Sciences Research Council (capital grant EP/T022159/1), and DiRAC funding from the Science and Technology Facilities Council (www.dirac.ac.uk).

References

[1] Leo Bachmair, Harald Ganzinger, David McAllester, and Christopher Lynch. Resolution Theorem Proving. In *Handbook of Automated Reasoning*, pages 19–99. Elsevier, 2001.

[2] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In *Computer Aided Verification*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152. Springer, 2024.

[3] Free Software Foundation. GNU lesser general public license, version 3, 2007.

[4] Michael Färber. A Curiously Effective Backtracking Strategy for Connection Tableaux, 2024.

[5] Michael Färber, Cezary Kaliszyk, and Josef Urban. Machine Learning Guidance for Connection Tableaux. 65(2):287–320, 2021.

[6] Cezary Kaliszyk, Josef Urban, Henryk Michalewski, and Miroslav Olšák. Reinforcement learning of theorem proving. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi,

and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, pages 8836–8847. Curran Associates, Inc., 2018.

[7] Reinhold Letz and Gernot Stenz. Model Elimination and Connection Tableau Procedures. In *Handbook of Automated Reasoning*, pages 2015–2114. Elsevier, 2001.

[8] Jens Otten. Restricting backtracking in connection calculi. 23(2–3):159–182, 2010.

[9] Jens Otten. 20 years of leanCoP — an overview of the provers. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023)*, volume 3613 of *CEUR Workshop Proceedings*, 2023.

[10] Jens Otten and Wolfgang Bibel. leanCoP: Lean connection-based theorem proving. 36(1–2):139–161, 2003.

[11] Jens Otten and Wolfgang Bibel. Advances in Connection-Based Automated Theorem Proving. In Mike Hinchey, Jonathan P. Bowen, and Ernst-Rüdiger Olderog, editors, *Provably Correct Systems*, pages 211–241. Springer International Publishing, 2017.

[12] Jens Otten and Sean Holden. A Syntax for Connection Proofs, 2023.

[13] Francis Jeffry Pelletier. Seventy-five problems for testing automatic theorem provers. 2(2):191–216, 1986.

[14] Michael Rawson, Clemens Eisenhofer, and Laura Kovács. Constraint Learning for Non-confluent Proof Search. In Gian Luca Pozzato and Tarmo Uustalu, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 15980, pages 103–119. Springer Nature Switzerland, 2026.

[15] Michael Rawson and Giles Reger. Eliminating Models During Model Elimination. In Anupam Das and Sara Negri, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 12842, pages 250–265. Springer International Publishing.

[16] Alexandre Riazanov and Andrei Voronkov. The design and implementation of VAMPIRE. 15(2–3):91–110, 2002.

[17] Fredrik Rømming, Jens Otten, and Sean B. Holden. Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*, volume 3613 of *CEUR Workshop Proceedings*, pages 107–118, 2023.

[18] Stephan Schulz. Light-weight integration of SAT solving into first-order reasoners: First experiments. In Laura Kovács and Andrei Voronkov, editors, *Vampire 2017. Proceedings of the 4th Vampire Workshop*, volume 53 of *EPiC Series in Computing*, pages 9–19. EasyChair, 2018.

[19] Stephan Schulz and Martin Möhrmann. Performance of Clause Selection Heuristics for Saturation-Based Theorem Proving. In Nicola Olivetti and Ashish Tiwari, editors, *Automated Reasoning*, volume 9706, pages 330–345. Springer International Publishing, 2016.

[20] Geoff Sutcliffe. The TPTP problem library and associated infrastructure: From CNF to TH0, TPTP v6.4.0. 59(4):483–502, 2017.

[21] University of Cambridge Research Computing Services. Ice Lake Nodes. https://docs.hpc.cam.ac.uk/hpc/user-guide/icelake.html, 2026. Accessed 13 May 2026.

[22] Zsolt Zombori, Josef Urban, and Chad E. Brown. Prolog Technology Reinforcement Learning Prover: (System Description). In Nicolas Peltier and Viorica Sofronie-Stokkermans, editors, *Automated Reasoning*, volume 12167, pages 489–507. Springer International Publishing, 2020.

[23] Zsolt Zombori, Josef Urban, and Miroslav Olšák. The Role of Entropy in Guiding a Connection Prover. In Anupam Das and Sara Negri, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 12842, pages 218–235. Springer International Publishing, 2021.